

Research Report: Designing a Modern Sample-Editing Application for Dance-Music Producers

Background and Market Landscape

Legacy inspiration – Sound Forge

Sound Forge was a popular destructive sample editor used by electronic musicians. Its user manual shows that the **Undo/Redo history window** lets users see all edit operations and audition “undo” and “redo” steps by clicking a play button ¹. Playback uses **real-time preview**: when the file is played in normal mode, **effects from the plug-in chain are previewed in real time** and can be bypassed ². Sound Forge also includes sample-loop modes where the file plays repeatedly according to loop settings and offers scrub controls, time zoom, zero-crossing editing and more ³. Jon Hopkins (electronic producer) explained why he used Sound Forge alongside Logic: he liked that it was **immediate and destructive**—“you change something and that’s it, it’s on the waveform” ⁴. He runs Sound Forge in a Windows virtual machine and edits audio files that appear inside Logic; edits performed in Sound Forge update the files in Logic automatically ⁵. Hopkins prints plug-in chains to disk and treats the resulting audio as an instrument ⁶; this workflow relies on destructive editing outside the DAW.

Competing sample-editing tools

Tool	Key features (relevant to sample editing)	Limitations
WaveLab (Steinberg)	The WaveLab manual describes crossfades: users can paste audio from one section into another and select crossfade types (linear, sinus, square-root) ⁷ . WaveLab supports time-stretching , pitch-shifting, spectral editing , multi-track “Audio Montage”, offline processing (normalize, DC-offset removal, reverse), multi-channel support and many processing presets ⁸ .	Designed primarily for mastering and audio-post work; crossfades and montage editing are oriented towards track compilation rather than creative sample manipulation. The interface is complex and may be overkill for quick sample manipulation.
Ocenaudio	A free cross-platform editor with VST plug-in support and real-time preview —effects (EQ, gain, filtering) can be adjusted while listening to processed audio ⁹ . It offers multi-selection editing (apply effects to multiple ranges at once) ¹⁰ , efficient editing of large files and a fully featured spectrogram view ¹¹ .	The feature set is basic; lacks advanced loop-editing tools and integration with DAWs.

Tool	Key features (relevant to sample editing)	Limitations
Audacity 3.7+	Open-source multi-track editor. The realtime effects panel (opened with the <i>Effects</i> button) allows adding VST3, VST, LV2, LADSPA or Audio-Unit plug-ins. Users can toggle effects on/off, bypass the entire stack, adjust settings in separate windows and apply master effects to the mixed output ¹² . Audacity offers plugin support and real-time preview but effects always apply to a whole track, not just a selection ¹³ .	Aimed at podcasts and general editing; lacks specialized loop-editing, crossfade-looping tools and integrated DAW workflow.
Acoustica (Acon Digital)	A professional editor with spectral and stereo editing, multi-track editing, high-quality processing tools and support for VST/AU plugins ¹⁴ . The interface allows docking meters, real-time analyzers and processing chains ¹⁵ . Premium edition adds multi-channel (Dolby Atmos) support and includes advanced plug-ins (Equalize 2, Verberate 2, Restoration Suite) ¹⁶ .	Targeted at mastering and restoration; less focused on quick loop manipulation.
General sample-editing functions (as summarised in a <i>Making Music</i> article)	Fundamental operations include cutting, pasting, snap to beat, zero-crossing points, DC-offset removal, crossfading, time-stretching, pitch shifting, phase inversion, reversing, normalisation and resampling . Many editors implement some of these functions differently, but they represent the baseline features expected in a sample editor.	Many consumer-oriented editors hide or simplify these features; specialized creative tools (e.g., crossfade-loop editing) are rare.

User needs and gaps

- **Real-time effect auditioning while looping** – Sound Forge previewed plug-in chains in real time ². Modern editors like Ocenaudio and Audacity have real-time previews, but they often apply effects to whole tracks (Audacity) or lack loop-length control. Dance-music producers often want to audition a sample through a chain of plug-ins (filters, saturation, reverb, delays) while adjusting parameters interactively.
- **Loop creation with reverb/delay tails** – Standard loop editors restrict loop length to the audio file. To include reverb or delay tails, producers often bounce the effect and manually extend the file; there is no simple way to define a loop length longer than the sample for auditioning.
- **Destructive editing outside the DAW** – Hopkin's workflow shows that destructive editing in Sound Forge allows instant changes that propagate to multiple occurrences in a DAW arrangement ⁵. Current editors rarely integrate with DAWs; bridging workflows via file

watchers or ARA (Audio Random Access) is emerging in tools like Acoustica but is not widely available for sample editing.

- **Plugin support and chain management** – Many editors support plug-ins but have limited chain management or no ability to reorder, bypass or audition entire chains. Real-time auditioning often suffers from latency or poor buffer management.
- **Creative processing vs. restoration/mastering focus** – Tools like WaveLab and Acoustica emphasize mastering, post-production and restoration, while many free editors target podcast editing. Dance-music producers need creative tools: filters, bit-crushers, granulators, sample-based modulation, etc.

Requirements to outstrip the market

Based on the research, a modern sample-editing application for dance-music producers should:

1. **Real-time effects auditioning** – Provide a plug-in chain with drag-and-drop ordering and the ability to add/bypass/modify plug-ins while the sample loops. Preview should be sample-accurate with minimal latency. Sound Forge's real-time preview of plug-in chains ² and Ocenaudio's real-time preview feature ¹⁷ demonstrate that this is achievable.
2. **Flexible loop length with tails** – Allow users to set the loop length independently of the sample length. The loop can extend beyond the sample to include reverb and delay tails; when loop audition is stopped, the tails continue playing.
3. **Creative effect modules** – Include built-in creative effects (filter sweeps, resonant EQ, LFO-modulated parameters, bit-crushers, granular processors) tailored for dance music. Support Audio Units and VST3 plug-ins so that third-party processors can be integrated. Ocenaudio demonstrates plug-in support ¹⁸ ; Audacity supports multiple plug-in formats ¹⁹ .
4. **Destructive and non-destructive workflows** – Provide two modes: *non-destructive* (stacked plug-in chain with undo history) and *destructive* (print the plug-in chain to disk and reset the chain). The undo/redo history should show all edits and allow auditioning prior states, similar to Sound Forge's undo/redo history with preview ²⁰ .
5. **Advanced loop editing** – Implement loop-point editing with crossfading like WaveLab. Allow crossfade types (equal-gain, equal-power, equal-power squared) and provide a "loop tuner" that plays the loop continuously while the user drags start/end points and crossfade lengths. The crossfade procedures described in WaveLab's manual ⁷ should inform the design.
6. **Spectral and zero-crossing views** – Include waveform and spectrogram views with zero-crossing markers. Provide tools for DC-offset correction, normalization and time/pitch manipulation as standard sample-editing functions.
7. **External DAW integration** – Offer file-watcher integration or an ARA-like bridge so that edits done in the sample editor update automatically in the user's DAW. Jon Hopkins' method of editing audio files in Sound Forge while they remain in Logic ⁵ is a powerful workflow. A plugin or script could watch for file changes and prompt the DAW to reload the audio.
8. **Multi-selection and batch operations** – Support selecting multiple regions in one file and applying edits to them simultaneously, as Ocenaudio does ¹⁰ . Batch processing of multiple files with the same effect chain would save time when creating sample libraries.
9. **10-level (or more) undo** – Provide a robust undo/redo system with at least 10 levels of undo. Sound Forge had a dedicated undo/redo history that allowed auditioning previous states ¹ .
10. **Universal Mac build** – Release as a Universal Binary supporting both ARM (Apple Silicon) and Intel Macs. Provide a roadmap for Windows support later.

Code-Factory-style development approach

Building such a sophisticated application can benefit from **AI-native “code factory” workflows**. A case-study from Fern Strategy describes code factories as multi-agent development systems that decompose features into tasks, run parallel engineer agents and gate output with compile and test validations ²¹. Key principles include:

- **Multi-agent decomposition** – Breaking a feature into tasks handled by specialized agents prevents context bloat ²². For example, separate agents could handle the audio engine (playback and looping), the plug-in chain UI, file I/O and DAW integration.
- **Spec coding & context management** – Generate detailed specifications and acceptance criteria up front. Each agent should have a clear goal and limited context, preventing large context windows from filling up ²³.
- **Validation layers** – After each task, run compile checks and automated tests; if a task fails, return to the engineer agent with error details ²⁴. This ensures that AI agents don’t hallucinate past errors.
- **Orchestrator and QA agents** – Use an orchestrator agent to manage the task DAG and a QA agent to run tests ²⁵. A spec-generation agent should produce acceptance criteria for each feature ²⁶.

Tools to use:

- **AI coding assistants** (e.g., Claude Code Opus 4.7, Cursor, Composer) integrated into VS Code. Use a harness (e.g., `devika`, `AutoGen` or custom) to spin up agents that can edit code, run tests and commit to a git repository. Ensure the harness supports persistent context and reproducible sessions.
- **Version control** – Each agent works in an isolated branch or git worktree; merges only occur when validation passes.
- **CI pipeline** – Run unit tests, integration tests and linter checks automatically on every agent submission.

To minimize supervision, invest time in writing **clear specifications** and **comprehensive tests** (see next section). The orchestrator should only ask for human intervention when tests consistently fail or when design decisions are unclear.

Example test specifications for AI agents

When you hand the project to an AI-powered coding team, provide tests that cover both functional requirements and user-experience goals. The following outline can serve as a starting point:

1. Core audio engine tests

1. **Loading and playing:** Given an audio file (e.g., a 10-second WAV), the engine should load it, play it back at normal speed and loop seamlessly when loop mode is enabled. When no loop points are set, the entire file should repeat; when loop points are set, only the selected section should loop.
2. **Extended loop length:** When the loop length is set to 15 seconds for a 10-second sample, the engine should play the sample once and continue playback for the remaining 5 seconds with silence or effect tail. If a plug-in chain adds reverb, the tail should be heard during the extended portion.

3. **Accurate seeking and zero-crossing:** Seek operations must land on zero-crossing positions; verifying that the sample starts at a zero amplitude reduces clicks. Tests can load a synthetic waveform with known zero-crossing positions and assert that selection rounding works.

2. Plug-in chain and real-time effects tests

1. **Plug-in loading:** The system should load built-in effects (EQ, filter, gain, fade in/out) and third-party Audio Unit/VST plug-ins from a test folder. A mock plug-in can expose parameters and output an identifiable tone (e.g., scales amplitude by 2). Tests should verify that the processed audio matches expectations when the plug-in is enabled.
2. **Real-time parameter changes:** While audio plays in a loop, changing a parameter (e.g., filter cutoff) should immediately alter the output. A test harness can automate parameter sweeps and capture output to ensure latency is below a threshold (e.g., < 50 ms).
3. **Chain ordering and bypass:** Reordering plug-ins should alter the resulting sound. Bypassing the entire chain should revert to the dry signal. Tests should compare rendered output with and without bypass.
4. **Printing to disk:** When the user chooses “print to disk,” the current processed buffer should be rendered to a new audio file. Tests should generate a short sample, apply an effect and then print; the resulting file’s waveform should match the live preview.

3. Loop and crossfade tests

1. **Crossfade loop creation:** Given a sample with a loop start/end and crossfade length, the system should apply an equal-power crossfade similar to WaveLab’s description ⁷. Tests can load a sine tone, create a loop and verify that there is no abrupt discontinuity at the loop point (no clicks).
2. **Loop tuner UI:** When the loop tuner is active, dragging the start/end points should update playback in real time. Automated UI tests can simulate dragging and assert that the playhead remains within the new loop boundaries.
3. **Sample-rate and tempo management:** When time-stretching or pitch-shifting is applied, the loop length and pitch should adjust accordingly. Tests can use a known tone and compare frequency and duration before and after processing.

4. Undo/redo and history tests

1. **Undo chain length:** Perform a sequence of 12 edits (normalise, reverse, fade out, etc.) and assert that the system retains at least the last 10 operations. Undoing should revert to previous states; redoing should reapply them. A preview of undo/redo states (e.g., by playing them) should be possible, mirroring Sound Forge’s history window ²⁰.
2. **Branching undo:** Undoing a mid-history operation and then performing a new edit should replace the subsequent redo history. Tests should confirm that the history list updates accordingly.

5. DAW integration tests (optional / advanced)

1. **File watcher:** Open a sample in both the sample editor and a simulated DAW (a dummy process reading the file). Edit the sample destructively in the editor; the watcher should detect the file change and notify the DAW, which reloads the sample.
2. **ARA plugin:** If implementing an ARA plug-in, tests should ensure that the plug-in can exchange audio data and edit state with DAW hosts (e.g., Logic, Studio One). This may require an integration harness or manual testing.

6. User interface tests

1. **Drag-and-drop effects:** Verify that dragging a plug-in into the chain adds it in the correct order. Dragging out removes it. Keyboard shortcuts should mirror these actions.
2. **Multi-selection:** Create three non-contiguous selections and apply normalization; only the selected regions should change. Undo should revert them.
3. **Spectrogram view:** Ensure that enabling spectrogram view does not interrupt audio playback and updates frequency bands in real time.

7. Performance and stability tests

1. **Large file handling:** Load and loop a several-hour audio file; editing operations (cut, copy, paste) should complete within a reasonable time and not freeze the UI. Ocenaudio's efficient editing of large files ²⁷ sets a benchmark.
2. **CPU usage during real-time effects:** Measure CPU and memory usage while stacking multiple plug-ins. Ensure that using moderate plug-in chains does not exceed a given CPU percentage on reference hardware.
3. **Crash recovery:** If the application crashes while editing, temporary files should allow restoring the previous state, similar to Sound Forge's crash recovery (temporary undo files) ²⁸.

Conclusion

Sound Forge's immediacy and real-time effect preview made it a favourite among electronic musicians like Jon Hopkins ⁴, but the software is dated and lacks modern integration. Contemporary editors like WaveLab, Ocenaudio, Audacity and Acoustica provide useful features—crossfades ⁷, real-time plug-in support ⁹ ¹³ and spectral editing—but are often aimed at mastering or podcasting rather than creative sample manipulation. A new sample editor targeting dance-music producers should combine real-time, non-destructive effect auditioning with flexible loop lengths, creative processing tools, robust undo history, DAW integration and a user-friendly interface. Adopting a **code factory** development approach—with multi-agent decomposition, clear specifications and automated validation—will enable an AI-driven coding team to build this complex application with limited supervision ²¹.

¹ ² ³ ²⁰ ²⁸ Sound Forge Pro 12.0 User Manual

https://shared.fastly.steamstatic.com/store_item_assets/steam/apps/821770/manuals/manual_soundforgepro12_en.pdf

⁴ ⁵ ⁶ King Creosote & Jon Hopkins: Recording Diamond Mine

<https://www.soundonsound.com/people/king-creosote-jon-hopkins-recording-diamond-mine>

⁷ ⁸ WaveLab Elements - Operation Manual

https://download.steinberg.net/downloads_software/WaveLab_Elements_8/Manuals/WaveLab-Elements-en-8.pdf

⁹ ¹⁰ ¹¹ ¹⁷ ¹⁸ ²⁷ ocenaudio

<https://www.ocenaudio.com/features>

¹² ¹³ ¹⁹ support.audacityteam.org

<https://support.audacityteam.org/audio-editing/using-realtime-effects>

¹⁴ ¹⁵ ¹⁶ Acoustica | Digital Audio Editor

<https://acondigital.com/products/acoustica>

²¹ ²² ²³ ²⁴ ²⁵ ²⁶ Fern Strategy - AI-Native Engineering Strategy & Agent Development

<https://www.fernstrategy.com/case-studies/code-factories>